

Analytics Vidhya

How to Determine the Optimal K for K-Means?



Khyati Mahendru

Follow

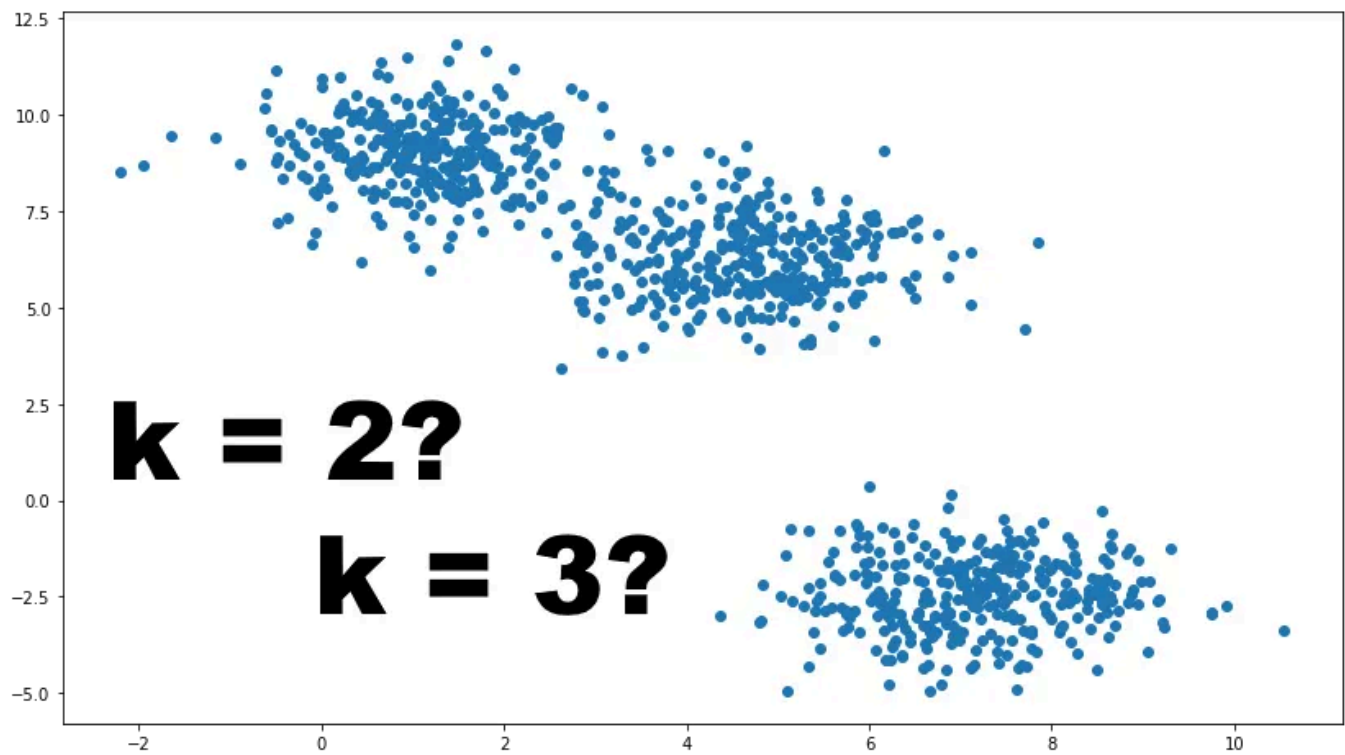
5 min read · Jun 17, 2019



1.1K



12



Introduction

The K-Means algorithm needs no introduction. It is simple and perhaps the most commonly used algorithm for clustering.

The basic idea behind k-means consists of defining k clusters such that total **within-cluster variation (or error) is minimum**.

I encourage you to check out the below articles for an in-depth explanation of different methods of clustering before proceeding further:

- [An Introduction to Clustering and different methods of Clustering](#)
- [A Beginner's Guide to Hierarchical Clustering and how to perform it in Python](#)

A cluster center is the representative of its cluster. The squared distance between each point and its cluster center is the required variation. The aim of k-means clustering is to find these k clusters and their centers while reducing the total error.

Quite an elegant algorithm. But there is a catch. How do you decide the number of clusters?

In this article, I will explain in detail two methods that can be useful to find this mysterious k in k-Means.

These methods are:

1. **The Elbow Method**
2. **The Silhouette Method**

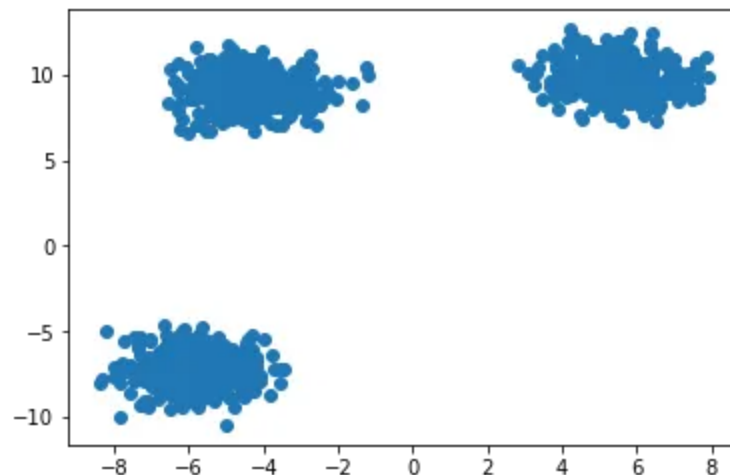
We will use our own dataset generated by the code below for an illustration of the two methods:

```
1 from sklearn.datasets import make_blobs
2
3 # Create dataset with 3 random cluster centers and 1000 datapoints
4 x, y = make_blobs(n_samples = 1000, centers = 3, n_features=2, shuffle=True, random_state=31)
```

generate_clustering_data.py hosted with ❤ by GitHub

[view raw](#)

This is how the data looks graphically:



Clearly, the dataset has 3 clusters. We will validate both our methods on this dataset.

The Elbow Method

This is probably the most well-known method for determining the optimal number of clusters. *It is also a bit naive in its approach.*

Calculate the Within-Cluster-Sum of Squared Errors (WSS) for different values of k , and choose the k for which WSS becomes first starts to diminish. In the plot of

| *WSS-versus-k, this is visible as an **elbow**.*

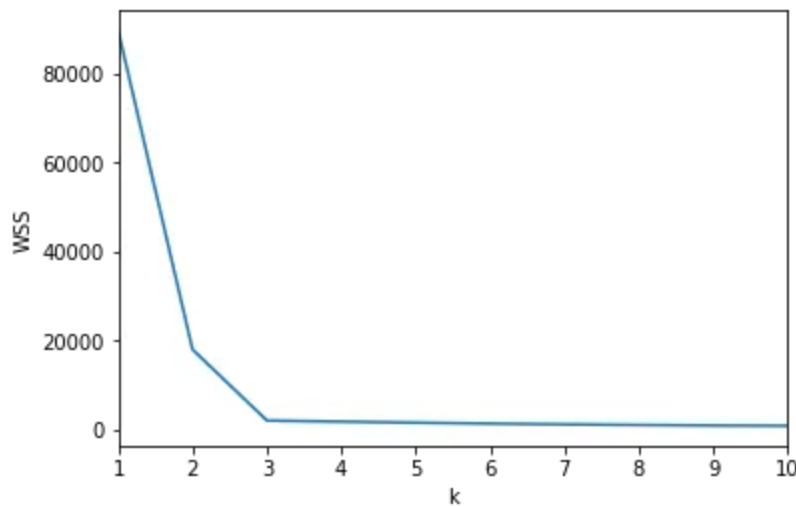
Within-Cluster-Sum of Squared Errors sounds a bit complex. Let's break it down:

- The Squared Error for each point is the square of the distance of the point from its representation i.e. its predicted cluster center.
- The WSS score is the sum of these Squared Errors for all the points.
- Any distance metric like the Euclidean Distance or the Manhattan Distance can be used.

Let us implement this in Python using the *sklearn* library and our own function for calculating WSS for a range of values for k.

```
1  from sklearn.cluster import KMeans
2
3  # function returns WSS score for k values from 1 to kmax
4  def calculate_WSS(points, kmax):
5      sse = []
6      for k in range(1, kmax+1):
7          kmeans = KMeans(n_clusters = k).fit(points)
8          centroids = kmeans.cluster_centers_
9          pred_clusters = kmeans.predict(points)
10         curr_sse = 0
11
12         # calculate square of Euclidean distance of each point from its cluster center and add to curr_sse
13         for i in range(len(points)):
14             curr_center = centroids[pred_clusters[i]]
15             curr_sse += (points[i, 0] - curr_center[0]) ** 2 + (points[i, 1] - curr_center[1]) ** 2
16
17         sse.append(curr_sse)
18     return sse
```

We obtain the following plot for WSS-vs-k for our dataset.



As expected, the plot looks like an arm with a clear elbow at $k = 3$.

Unfortunately, we do not always have such clearly clustered data. This means that the elbow may not be clear and sharp.



Source: blocks.org/

For Dataset A, the elbow is clear at $k = 3$. However, this choice is ambiguous for Dataset B. We could choose k to be either 3 or 4.

In such an ambiguous case, we may use the Silhouette Method.

The Silhouette Method

The silhouette value measures how similar a point is to its own cluster (cohesion) compared to other clusters (separation).

Source: Wikipedia

The range of the Silhouette value is between +1 and -1. A **high value is desirable** and indicates that the point is placed in the correct cluster. If many points have a negative Silhouette value, it may indicate that we have created too many or too few clusters.

The Silhouette Value $s(i)$ for each data point i is defined as follows:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \text{ if } |C_i| > 1$$

and

$$s(i) = 0, \text{ if } |C_i| = 1$$

Source: Wikipedia

Note: $s(i)$ is defined to be equal to zero if i is the only point in the cluster. This is to prevent the number of clusters from increasing significantly with many single-point clusters.

Here, $a(i)$ is the measure of similarity of the point i to its own cluster. It is measured as the average distance of i from other points in the cluster.

For each data point $i \in C_i$ (data point i in the cluster C_i), let

$$a(i) = \frac{1}{|C_i| - 1} \sum_{j \in C_i, i \neq j} d(i, j)$$

Source: Wikipedia

Similarly, $b(i)$ is the measure of dissimilarity of i from points in other clusters.

For each data point $i \in C_i$, we now define

$$b(i) = \min_{i \neq j} \frac{1}{|C_j|} \sum_{j \in C_j} d(i, j)$$

Source: Wikipedia

$d(i, j)$ is the distance between points i and j . Generally, **Euclidean Distance** is used as the distance metric.

The Silhouette score can be easily calculated in Python using the metrics module of the *sklearn* library.

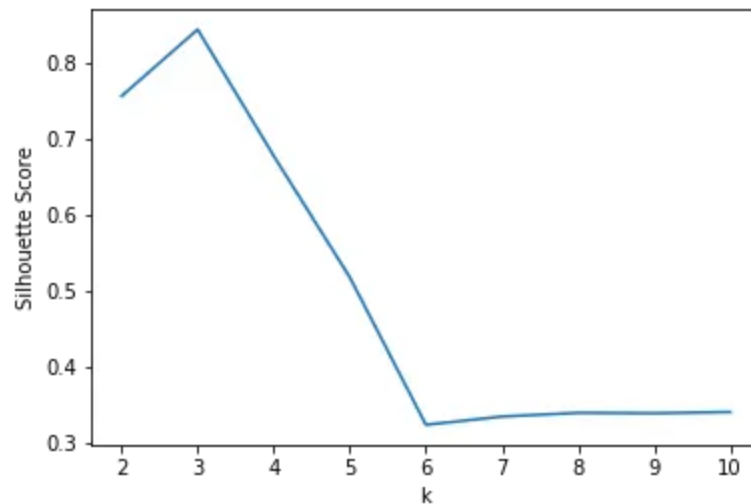
```
1  from sklearn.metrics import silhouette_score
2
3  sil = []
4  kmax = 10
5
6  # dissimilarity would not be defined for a single cluster, thus, minimum number of clusters should
7  for k in range(2, kmax+1):
8      kmeans = KMeans(n_clusters = k).fit(x)
9      labels = kmeans.labels_
10     sil.append(silhouette_score(x, labels, metric = 'euclidean'))
```

silhouette_score.py hosted with ❤ by GitHub

[view raw](#)

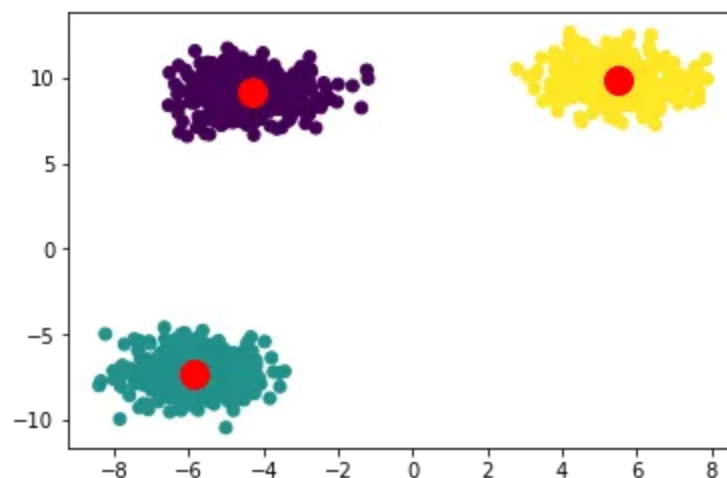
I mentioned before that a high Silhouette Score is desirable. The Silhouette Score reaches its *global maximum at the optimal k*. This should ideally appear as a peak in the Silhouette Value-versus-k plot.

Here is the plot for our own dataset:



There is a clear peak at $k = 3$. Hence, it is optimal.

Finally, the data can be *optimally* clustered into 3 clusters as shown below.



End Notes

The Elbow Method is more of a decision rule, while the Silhouette is a metric used for validation while clustering. Thus, it can be used in combination with the Elbow Method.

Therefore, the Elbow Method and the Silhouette Method are not alternatives to each other for finding the optimal K. Rather they are tools to be used together for a more confident decision.

Machine Learning

Data Science

Analytics Vidhya

Clustering

Unsupervised Learning



Published in Analytics Vidhya

76K followers · Last published Jul 16, 2025

Analytics Vidhya is a community of Generative AI and Data Science professionals. We are building the next-gen data science ecosystem
<https://www.analyticsvidhya.com>

Follow



Written by Khyati Mahendru

424 followers · 22 following

SWE @ Google | Mathematics and Computing
<https://www.linkedin.com/in/khyati-mahendru/>

Follow

Responses (12)





Md Ashfak Uzzaman Chowdhury

What are your thoughts?



Konstantina Tsekoura

Feb 1, 2020 (edited)



Thank you for your article :)

What if i have to choose number of clusters in an automated way? Could i select the silhouette_score?

And also, what if the elbow method gives me different number of cluster from the silhouette_score method?



3



1 reply

[Reply](#)



Leon Wang

Mar 3, 2022



Silhousette metric may be used for small datasets. If the datasets have large number, the decisioin process would be very long.

Open in app ↗



Medium



Search



Write

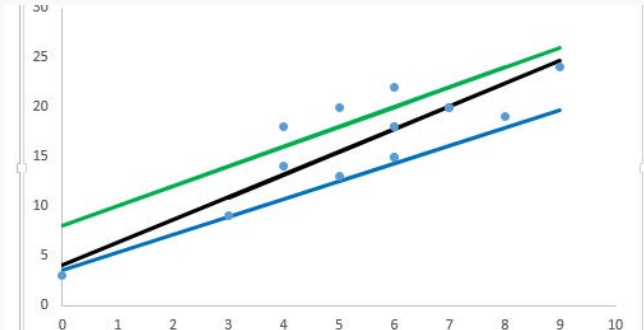


1

[Reply](#)

See all responses

More from Khyati Mahendru and Analytics Vidhya



In Analytics Vidhya by Khyati Mahendru

Measuring the Goodness of Fit: R^2 versus Adjusted R^2

Linear Regression deals with modeling a linear relationship between a dependent...

Jun 21, 2019 260 3



In Analytics Vidhya by Harikrishnan N B

Confusion Matrix, Accuracy, Precision, Recall, F1 Score

Binary Classification Metric

Dec 11, 2019 1.3K 7





 In Analytics Vidhya by Leland Roberts

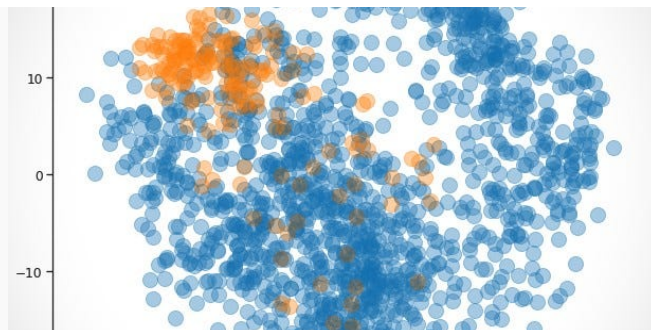
Understanding the Mel Spectrogram

(and Other Topics in Signal Processing)

Mar 6, 2020

 2.3K

 28



 In Analytics Vidhya by Khyati Mahendru

How to Deal with Imbalanced Data using SMOTE

With a Case Study in Python

Jun 25, 2019

 1K

 7



See all from Khyati Mahendru

See all from Analytics Vidhya

Recommended from Medium



 In Python in Plain English by Mohamad Mahmood



 Kyle Jones 

Text Clustering: Comparing TF-IDF, BERT, and SBERT Embeddings wi...

Text clustering is a fundamental task in natural language processing that aims to...

★ Mar 17 🤝 3



$$(x, \mu) = \sqrt{\sum_{i=1}^n (x_i - \mu_i)^2}$$

Avinash Mohan

Outlier Analysis, Outlier Scores, K-Means Clustering

🔑 Key Insight:

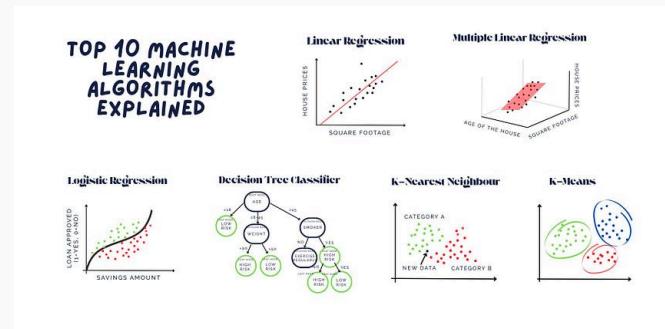
★ Jun 21



Predictive Maintenance with Ensemble Models for Ordered Ti...

Forecasting ordered outcomes over time demands flexibility. Some models capture...

★ Apr 1 🤝 21 💬 2

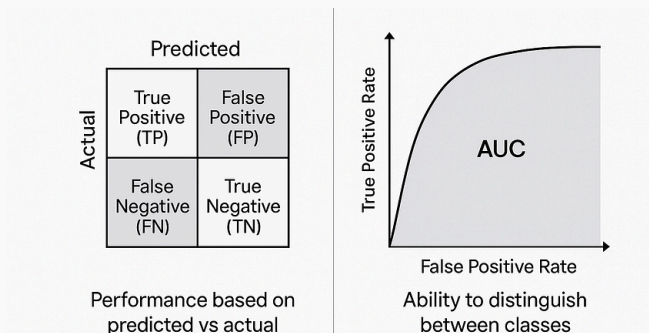


In Learning Data by Rita Angelou

10 ML Algorithms Every Data Scientist Should Know—Part 1

I understand well that machine learning might sound intimidating. But once you break dow...

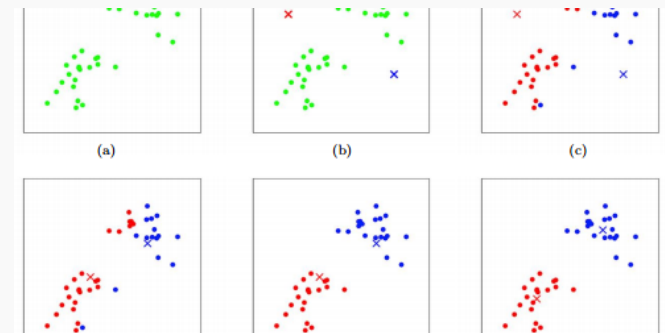
★ Jun 10 🤝 29



GunKurnia

Understanding Confusion Matrix vs. AUC-ROC Curve: Essential Too...

The Confusion Matrix offers a detailed snapshot of a model's prediction accuracy b...



Andres Felipe Tellez Yepes

Understanding Clustering: K-Means, Hierarchical, DBSCAN

Series on becoming a better Data Scientist

★ Apr 8 🖱️ 14



Apr 9



See more recommendations